Practice Interview

Objective

The partner assignment aims to provide participants with the opportunity to practice coding in an interview context. You will analyze your partner's Assignment 1. Moreover, code reviews are common practice in a software development team. This assignment should give you a taste of the code review process.

Group Size

Each group should have 2 people. You will be assigned a partner

Parts:

- Part 1: Complete 1 of 3 questions
- Part 2: Review your partner's Assignment 1 submission
- Part 3: Perform code review of your partner's assignment 1 by answering the questions below
- Part 3: Reflect on Assignment 1 and Assignment 2

Part 1:

You will be assigned one of three problems based of your first name. Enter your first name, in all lower case, execute the code below, and that will tell you your assigned problem. Include the output as part of your submission (do not clear the output). The problems are based-off problems from Leetcode.

In [1]:
```
import hashlib

def hash_to_range(input_string: str) -> int:
     hash_object = hashlib.sha256(input_string.encode())
     hash_int = int(hash_object.hexdigest(), 16)
```

```
    return (hash_int % 3) + 1
input_string = "simon"
result = hash_to_range(input_string)
print(result)
```

```
2
```

▶ Question 1

Starter Code for Question 1

In [ ]:
```
# Definition for a binary tree node.
# class TreeNode(object):
#     def __init__(self, val = 0, left = None, right = None):
#         self.val = val
#         self.left = left
#         self.right = right
def is_duplicate(root: TreeNode) -> int:
  # TODO
```

▶ Question 2

Starter Code for Question 2

In [ ]:
```
# Definition for a binary tree node.
# class TreeNode(object):
#     def __init__(self, val = 0, left = None, right = None):
#         self.val = val
#         self.left = left
#         self.right = right
def bt_path(root: TreeNode) -> List[List[int]]:
  # TODO
```

In [ ]:
```
class TreeNode(object):
    def __init__(self, val = 0, left = None, right = None):
        self.val = val
        self.left = left
        self.right = right


def bt_path(root: TreeNode):
```

```
    if root is None: # base case for root
        return []

    if root.left is None and root.right is None: # base case for leaf
        return[[root.val]]

    path = []

    for p in bt_path(root.left): # collect the left path 1st
        #print(root.left.val, root.val)
        path.append([root.val] + p)

    for p in bt_path(root.right): # then collect the right path
        #print(root.right.val, root.val)
        path.append([root.val] + p)

    return path # expected return the path in list
```

In [ ]:
```
# eveluate

# Build the example tree
# Input: root = [1, 2, 2, 3, 5, 6, 7]
# Output: [[1, 2, 3], [1, 2, 5], [1, 2, 6], [1, 2, 7]]

bst = TreeNode(1)
bst.left = TreeNode(2)
bst.right = TreeNode(2)
bst.left.left = TreeNode(3)
bst.left.right = TreeNode(5)
bst.right.left = TreeNode(6)
bst.right.right = TreeNode(7)
bst

print(bt_path(bst))
```

```
[[1, 2, 3], [1, 2, 5], [1, 2, 6], [1, 2, 7]]
```

In [ ]:
```
# Input: root = [10, 9, 7, 8]
# Output: [[10, 7], [10, 9, 8]]

bst2 = TreeNode(10)
bst2.left = TreeNode(9)
```

```
bst2.right = TreeNode(7)
bst2.left.left = TreeNode(8)
bst2

print(bt_path(bst2))
```

```
[[10, 9, 8], [10, 7]]
```

▶ Question 3

Starter Code for Question 3

In [ ]:
```
def missing_num(nums: List) -> int:
  # TODO
```

Part 2:

You and your partner must share each other's Assignment 1 submission.

Part 3:

Create a Jupyter Notebook, create 6 of the following headings, and complete the following for your partner's assignment 1:

- Paraphrase the problem in your own words.

In [ ]:
```
# Your answer here
Paraphrase Question One

########
Given a list of integers.
Write a function to find the first integer that appears more than once in the list.

The function should follow these rules:
- An integer is considered duplicated if it appears at least two times in the list.
- If multiple integers have duplicates, return the integer whose duplicate occurrence is found first
  when scanning the list from left to right.
- If no duplicate integer exists, return -1.
```

- Create 1 new example that demonstrates you understand the problem. Trace/walkthrough 1 example that your partner made and explain it.

In [ ]:
```
# Your answer here

My example
Input: nums = [4, 7, 8, 9, 8, 2, 3]
Output: 8

My partner example
Input: nums = [10, 2, 5, 2, 10, 5, 7]
Output: 2

Scan the list from left to right:

10 first appears at index 0.
2 first appears at index 1.
5 first appears at index 2.
2 appears again at index 3, so it is the first duplicated integer found.
10 appears again at index 4, so it is also duplicated, but is the 2nd duplicated integer.
5 appears again at index 5, so it is also duplicated, but is the 3rd duplicated integer.
7 first appears at index 6.

Therefore, the first duplicate value encountered is 2 and duplicate occurrence is at index 3,
so the output is 2.
```

- Copy the solution your partner wrote.

In [5]:
```
# Your answer here
from typing import List

def first_duplicate(nums: List[int]) -> int:
    seen = set()
    for item in nums:
        if item in seen:
            return item
        else:
```

```
            seen.add(item)
    return -1
```

In [6]:
```
first_duplicate([10, 2, 5, 2, 10, 5, 7])
```

Out[6]: 2

In [7]:
```
first_duplicate([2,1,1,2,1])
```

Out[7]: 1

- Explain why their solution works in your own words.

In [ ]:
```
# Your answer here

The function scans the list from left to right and stores each new number in a set called seen.
The set keeps track of unique numbers that have already appeared in the list.

When the function encounters a number that already exists in seen, it returns that number as the
first duplicate found.

If the entire list is scanned and no duplicate number is found, the function returns -1.
```

- Explain the problem's time and space complexity in your own words.

In [ ]:
```
# Your answer here

In the worst-case scenario, where the list contains no duplicate values, the function must scan through
all n elements in the list once. Each operation, such as checking whether a number exists in the set
and adding a new number, takes constant time. Therefore, the overall time complexity is O(n).

For space complexity, each new unique number is stored in the seen set. In the worst case,
all n numbers are unique, so the set grows to contain n elements. Therefore, the space complexity is O(n).
```

- Critique your partner's solution, including explanation, and if there is anything that should be adjusted.

In [ ]:
```
# Your answer here

if item in seen:
            return item
        else:
            seen.add(item)

The "else" can be removed because the code has only two possible outcomes: either a duplicate is found and returned,
or the new number is added to the "seen" set.
```

Part 4:

Please write a 200 word reflection documenting your process from assignment 1, and your presentation and review experience with your partner at the bottom of the Jupyter Notebook under a new heading "Reflection." Again, export this Notebook as pdf.

Reflection

In [ ]:
```
# Your answer here

In Assignment 1, I learned how to use a stack to solve the Valid Parentheses problem.
I started by creating a variety of random bracket sequences to identify common patterns and uncover edge cases
involving unmatched or incorrectly ordered brackets. One of the main challenges I faced was handling nested
brackets correctly and ensuring that every opening bracket was matched with the appropriate closing bracket.
I also had to consider several edge cases, such as empty strings and incomplete bracket sequences.

The main challenge I faced in Assignment 2 was understanding how recursion works with tree structures.
Initially, I needed to add debugging code to visualize how each recursive call moves through the tree and
returns back to the previous node. I also had to carefully handle edge cases, such as an empty tree or a
tree containing only a single node.

During my presentation, I explained how a dictionary can be used to efficiently map opening and closing brackets,
allowing the algorithm to quickly determine whether a pair matches. I also described how the stack serves as
temporary storage for opening brackets until their corresponding closing brackets are encountered.
Finally, I explained the algorithm's time and space complexity and why both are O(n) in the worst case.

While reviewing my partner's work, I noticed that the term "First Duplicate" can be somewhat ambiguous and easy
to misunderstand. I spent some time researching its meaning in computer science to better understand the problem.
```

```
Overall, my partner's code was clean and well organized. The only improvement I suggested was removing an
unnecessary else statement to make the code slightly simpler.
```

Evaluation Criteria

We are looking for the similar points as Assignment 1

- Problem is accurately stated
- New example is correct and easily understandable
- Correctness, time, and space complexity of the coding solution
- Clarity in explaining why the solution works, its time and space complexity
- Quality of critique of your partner's assignment, if necessary

Submission Information

🚨 **Please review our Assignment Submission Guide** 🚨 for detailed instructions on how to format, branch, and submit your work. Following these guidelines is crucial for your submissions to be evaluated correctly.

Submission Parameters:

- Submission Due Date: `HH:MM AM/PM - DD/MM/YYYY`
- The branch name for your repo should be: `assignment-2`
- What to submit for this assignment:
  - This Jupyter Notebook (assignment_2.ipynb) should be populated and should be the only change in your pull request.
- What the pull request link should look like for this assignment:
  `https://github.com/<your_github_username>/algorithms_and_data_structures/pull/<pr_id>`
  - Open a private window in your browser. Copy and paste the link to your pull request into the address bar. Make sure you can see your pull request properly. This helps the technical facilitator and learning support staff review your submission easily.

Checklist:

- [ ] Created a branch with the correct naming convention.
- [ ] Ensured that the repository is public.
- [ ] Reviewed the PR description guidelines and adhered to them.
- [ ] Verify that the link is accessible in a private browser window.

If you encounter any difficulties or have questions, please don't hesitate to reach out to our team via our Slack at `#cohort-6-help`. Our Technical Facilitators and Learning Support staff are here to help you navigate any challenges.

In [ ]: